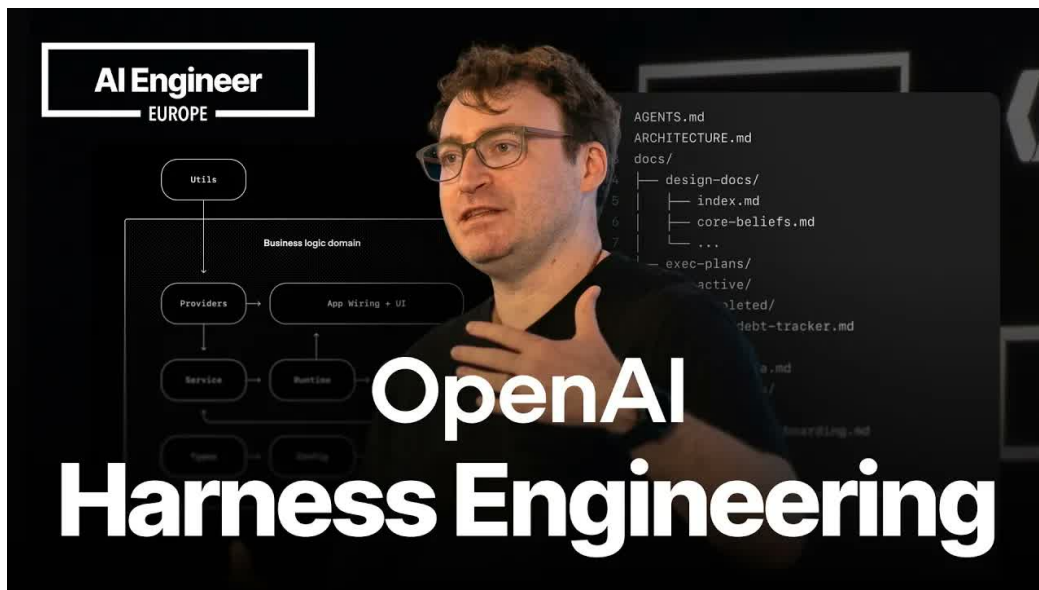


# Harness Engineering: 当人类掌舵、Agent 执行时，软件工程如何重构

SSS & Codex

2026-07-05



视频作者/频道: AI Engineer

发布日期: 2026-04-17

视频时长: 46:20

视频链接: [https://www.youtube.com/watch?v=am\\_oeAoUheW](https://www.youtube.com/watch?v=am_oeAoUheW)

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

# 目录

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>引言：从写代码转向设计执行环境</b>                              | <b>2</b>  |
| <b>1 实现不再稀缺：code is free 带来的工程角色变化</b>              | <b>2</b>  |
| 1.1 结论：实现能力变成可调度资源                                  | 2         |
| 1.2 动机：从 token billionaire 到团队执行容量                  | 2         |
| 1.3 机制：新稀缺资源重排工程角色                                  | 3         |
| 1.4 例子与证据：P3、国际化、迁移和 500 个小决策                       | 4         |
| 1.5 风险与边界：免费代码仍会产生 slop                             | 5         |
| 1.6 本章小结                                            | 6         |
| <b>2 Harness 的基本构件：把“好工作”写成 agent 可读的系统</b>         | <b>6</b>  |
| 2.1 从个人经验到团队级上下文资产                                  | 6         |
| 2.2 上下文会分页，所以约束必须持续刷新                               | 7         |
| 2.3 四类构件：文档、规则、检查器、执行工具                             | 7         |
| 2.4 好的错误消息要能指导下一步修复                                 | 8         |
| 2.5 Everything is a prompt：提示面分布在工程系统里              | 8         |
| 2.6 QA plan：把产品验收也写进 Harness                        | 9         |
| 2.7 本章小结                                            | 9         |
| <b>3 从 ticket 到 PR：Codex-first 工作流与协作面</b>          | <b>10</b> |
| 3.1 从 ticket 开始：把工作切成 agent 可接收的任务                  | 10        |
| 3.2 Codex 调用链：skills 把本地工具变成 agent 接口               | 11        |
| 3.3 本地 guardrails：custom ESLint rules 与源码结构验证       | 11        |
| 3.4 最低充分上下文：在正确时间给模型正确文本                            | 12        |
| 3.5 first-party harnesses：post-training 带来的工具语义杠杆   | 12        |
| 3.6 PR 作为 hub-and-spoke broadcast domain            | 12        |
| 3.7 本章小结                                            | 13        |
| <b>4 把仓库改造成上下文机器：结构、一致性与 progressive disclosure</b> | <b>13</b> |
| 4.1 个人迁移路径：先补测试，再审计时间                               | 14        |
| 4.2 无人值守任务：tests are green 是最低完成信号                  | 14        |
| 4.3 progressive disclosure：上下文按时机释放                 | 15        |
| 4.4 从单包混乱到 PNPM workspace：让边界可见                     | 15        |
| 4.5 make things the same：一致性降低注意力成本                 | 16        |
| 4.6 隐含依赖与风险边界                                       | 16        |
| 4.7 本章小结                                            | 17        |
| <b>5 反馈闭环：从 code review 到自动 guardrails</b>          | <b>17</b> |
| 5.1 结论：把人工评审转成长期约束                                  | 17        |
| 5.2 动机：高 PR 速度会放大同步成本                               | 17        |
| 5.3 garbage collection day：把一周的 slop 分类             | 18        |
| 5.4 机制：从评论到文档、测试和 reviewer agents                   | 18        |
| 5.5 Persona reviewer：只浮出能阻塞合并的问题                    | 19        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 5.6      | 可迁移模板：从一次评审到长期 guardrail . . . . .                            | 19        |
| 5.7      | 本章小结 . . . . .                                                | 20        |
| <b>6</b> | <b>总结与延伸：LLM as fuzzy compiler 与长期 autonomous engineering</b> | <b>20</b> |
| 6.1      | 结论：未来的软件工程输入会更像约束、指标和验收标准 . . . . .                           | 20        |
| 6.2      | Token 预算：大致三分，避免精确会计 . . . . .                                | 20        |
| 6.3      | plan mode 与 exact plans：计划本身也是代码路径 . . . . .                  | 21        |
| 6.4      | 从 accepted code 到 senior engineers as agents . . . . .        | 21        |
| 6.5      | LLM as fuzzy compiler：把仓库上下文看作约束和优化 pass . . . . .            | 22        |
| 6.6      | 长期 autonomous engineering：把软件工程外延纳入 agent 闭环 . . . . .        | 23        |
| 6.7      | 全文结构化提炼：六章共同回答一个问题 . . . . .                                  | 24        |
| 6.8      | 本章小结 . . . . .                                                | 24        |

# 导言：从写代码转向设计执行环境

这份讲义整理自 Ryan Lopopolo 在 AI Engineer Europe 的演讲与后续 Q&A。视频前半段提出 **Harness Engineering** 的核心主张，后半段通过现场问答补足实际 workflow、团队组织、代码评审、token 使用和未来愿景。全文按教学主题重组材料，主线是：当 **humans steer, agents execute** 成为软件开发的新分工，工程师的高杠杆工作会从亲自写实现，转向定义目标、暴露上下文、设置 guardrails、安排 reviewer agents，并把反复出现的人工判断沉淀为 agent 可执行的工程环境。

## 全文核心结论

**Harness Engineering** 可以理解为“给 coding agents 使用的软件工程操作系统”：它把任务、文档、技能、工具、测试、评审、CI、PR 协作和验收标准组织成一个可执行环境。模型能力越强，代码生成越便宜，团队越需要把人的判断写成可见、可检查、可复用的约束。

## 1 实现不再稀缺：code is free 带来的工程角色变化

### 1.1 结论：实现能力变成可调度资源

Ryan Lopopolo 在开场给出的核心判断是：**Harness Engineering** 的起点来自稀缺资源的变化。当 coding agents 能够持续读仓库、改代码、运行工具、响应反馈时，软件工程中的“实现”开始变成可调度资源；工程师的高杠杆工作随之转向定义目标、表达约束、安排验证、建立 **guardrails**，并让多个 agent 在这些约束中执行。视频标题里的 **humans steer, agents execute** 正是在概括这层分工：人类负责方向、优先级、边界和验收，agent 负责实现、重构、修复和反复试错。

## 本章主结论

这里的 **code is free** 应理解为代码生产、重构、删除的边际执行成本显著下降。它并不取消产品价值、运行风险、维护负担和团队理解成本。正因为代码更容易生成，工程师更需要把“什么算好工作”写成 agent 可见、可执行、可检查的系统。

这也是第一章的入口：如果实现能力已经丰富，亲手敲出每一行代码的占比下降，工程师剩下的关键能力集中到判断哪些工作值得做、怎样拆解工作、怎样让上下文进入模型、怎样阻止低质量输出进入主干，以及怎样把一次人工反馈沉淀成长期复用的工程约束。

### 1.2 动机：从 token billionaire 到团队执行容量

Ryan 先自己的工作状态建立可信背景：他在 OpenAI 过去 9 个月主要通过 agents 构建软件，并把自己称为 **token billionaire**。这个说法的教学意义不在夸张的 token 数字，而在提示一种新的资源观：输出 token、GPU capacity、agent 并行度和自动化执行时间，正在成为工程团队可以调度的容量。

在传统软件团队里，增加实现能力通常意味着招聘更多“hands on keyboards”。在 Ryan 的叙述中，coding agents 改变了这个限制条件：一个工程师可以同时驱动 5 个、50 个，甚至更多并行执行者。此时，真正困难的部分变成如何把这些执行者部署到正确的问题上，并让它们在真实代码库中完成可接受的工作。

## staff engineer 类比

Ryan 说“每个人都像 staff engineer”，重点在角色重心变化。staff engineer 的典型价值来自系统设计、任务拆解、跨团队协调、标准制定和长期技术方向。agent 时代把这种工作方式下放给更多工程师：单个工程师需要面向 1 天、1 周、6 个月之后思考，提前准备结构、文档、测试和约束，使多个 agent 能够并行推进。

这种变化也解释了为什么主题叫 **Harness Engineering**。harness 原本有“把力量套入可控结构”的含义。放到软件工程中，它指把 agent 的实现能力接入真实团队流程的系统：任务说明、仓库结构、文档、测试、lint、review agents、CI 和验收标准共同决定 agent 能否完成“全套工作”。

### 1.3 机制：新稀缺资源重排工程角色

Ryan 在第一组核心幻灯片中把这套判断压缩成三个命题：模型足够强，**code is free**，工程师的角色是释放团队能力。它们组成一个因果链：模型能力提升让实现供给增加；实现供给增加让代码本身不再是主要瓶颈；瓶颈迁移之后，人类必须把注意力放到系统、边界和验收上。

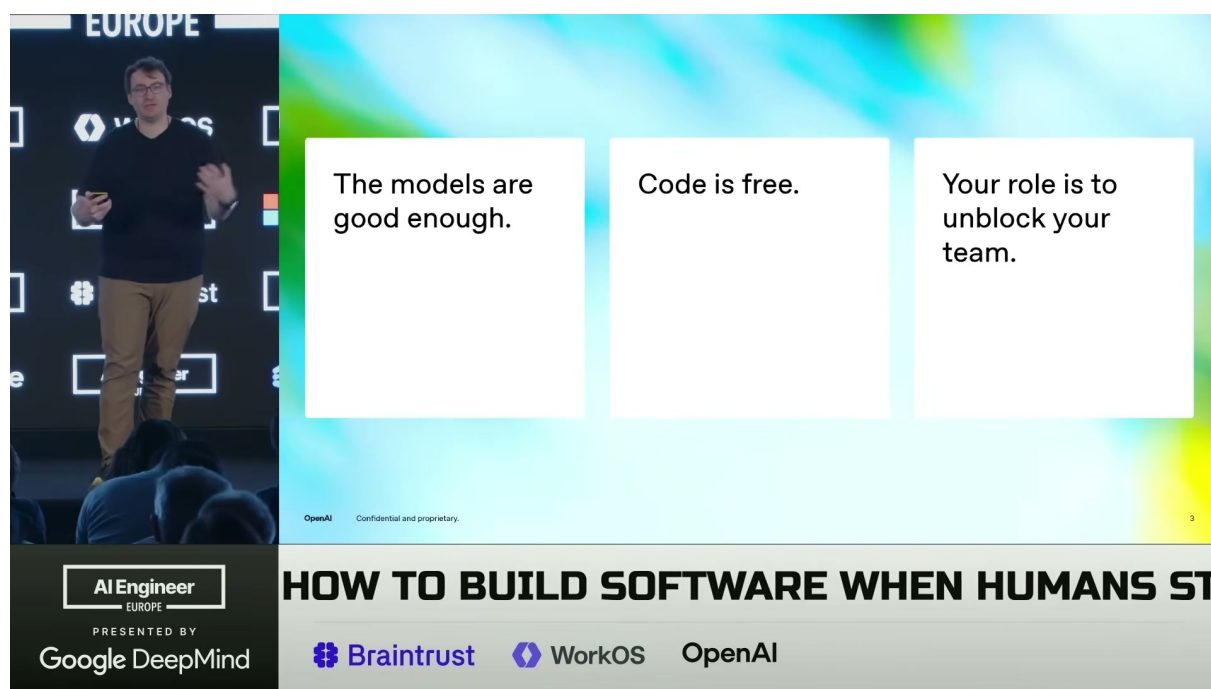


图 1: Ryan 用三栏幻灯片给出开场总论：模型已经足够强，*code is free*，工程师的职责转向释放团队能力。<sup>1</sup>

这张图中的三句话需要连起来理解。第一句“模型足够强”是能力前提，Ryan 将其归因于 coding agent 在更长时间范围内执行复杂任务的可靠性提升。第二句 **code is free** 是成本判断，强调生产、维护、重构和删除代码时，人类同步注意力不再总是关键约束。第三句“释放团队能力”是组织结论：工程师要让 agent 和人类团队都能在更少阻塞中完成高质量工作。

随后的稀缺资源图把推理推进了一层：代码供给丰富之后，真正稀缺的是 **human time**、**human and model attention**、**model context window**。这三项分别对应软件工程中的三个瓶颈：人类

<sup>1</sup> 视频画面时间区间：00:02:59–00:04:44。

不能同步盯住所有任务，模型不能同时关注无限上下文，仓库和任务信息也不能无限塞进同一个上下文窗口。

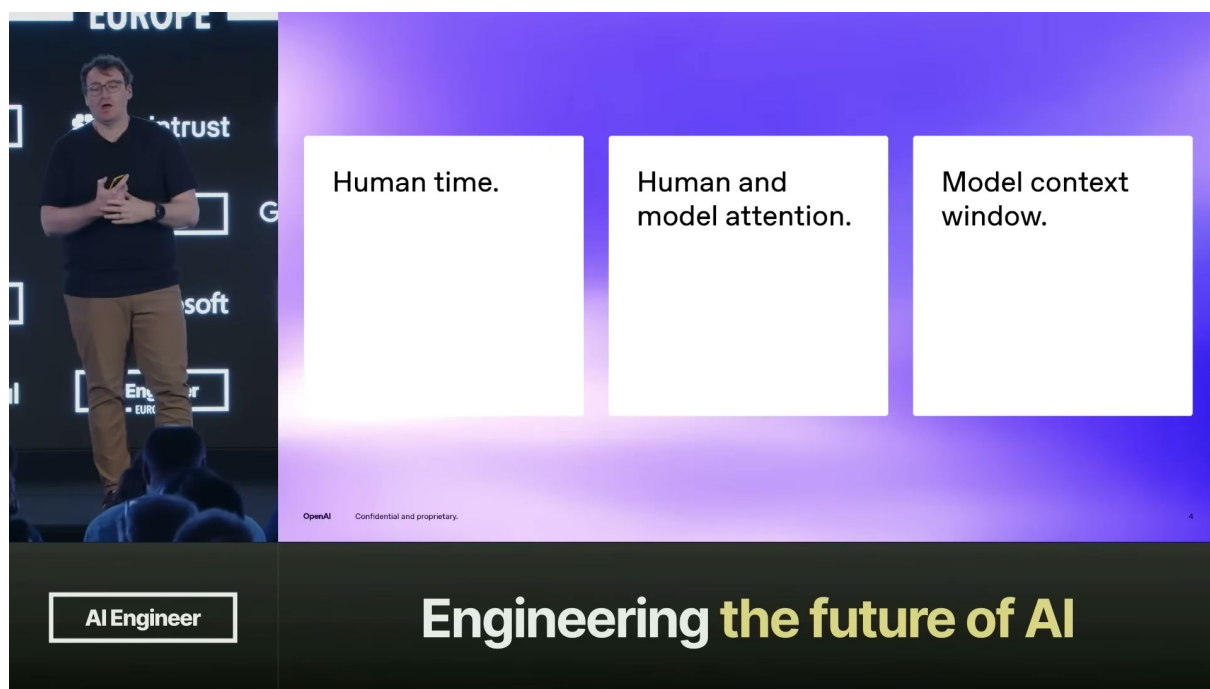


图 2: 当实现供给变得充足，稀缺资源转移到人的时间、人与模型的注意力，以及模型的 *context window*。<sup>2</sup>

这三种稀缺资源对应三类工程动作：

- **节省 human time**：把等待、重复实现、低信号 review、机械迁移交给 agent，把人类时间转移到判断、设计、验收和协调。
- **节省 attention**：通过一致的代码结构、稳定的工具入口、明确的任务说明和自动检查，减少人类与模型反复搜索、猜测和纠偏。
- **节省 context window**：把长期知识写进仓库文档、规则、测试和错误消息，让 agent 在需要时获得相关上下文，避免一次性灌入过量背景。

因此，**Harness Engineering** 的机制可以概括为：把“好工作”的隐性经验转化为 agent 可见的上下文和可执行的检查。代码生成能力越强，这套机制越重要，因为高速度会放大模糊需求、隐性标准、弱测试和低质量输出。

## 1.4 例子与证据：P3、国际化、迁移和 500 个小决策

第一个例子是优先级结构变化。这里的 P0、P2、P3 可以按工程团队常见优先级读法理解：P0 是必须立即处理的阻塞性工作，例如生产事故、安全风险、核心用户路径中断或收入相关故障；P2 是重要且应进入近期排期的工作，例如可靠性改进、关键体验修复、重要内部工具能力或会影响后续开发效率的工程质量问题；P3 是价值存在但紧急性较低的尾部改善，例如小型内部工具增强、体验打磨、低风险重构、非核心地区本地化覆盖或多条备选实现探索。Ryan 在演讲中没有展开正式优先级框架，他借这组标签说明一种资源分配现象：传统团队里，人类实现时间稀缺，工作经常被压成 P0 和 P2；P3 类型的低优先级改善长期排队，甚至长期搁置。Ryan 的说法是，在 **code is free**

<sup>2</sup> 视频画面时间区间：00:04:44–00:05:21。



的世界里，这些 P3 工作可以被并行启动，多条实现路径同时试跑，最后保留解决问题的一条。这里的关键在于边际试错成本下降以后，团队可以更早探索以前没有资源探索的改善；这并不表示所有低优先级工作都值得做。

第二个例子是内部工具的 localization 和 internationalization。Ryan 提到，给 OpenAI 内部同事构建生产力工具时，面向伦敦、都柏林、巴黎、布鲁塞尔、苏黎世、慕尼黑等地区的本地化体验可以从第一天就纳入工具质量要求。过去这类工作常被当作“以后再补”的体验债；当 agent 执行容量充足时，它可以变成默认的产品完整性要求。

第三个例子是长期 migration。Ryan 认为，大规模重构和一致化在 agent 时代更容易完成，因为团队可以同时派出多个 agent 处理不同子树、包或模块。迁移不再必然拖成 6 个月的悬空工作。它仍需要清晰目标、边界、测试和 review，但执行层面的尾部工作更容易被推到完成。

#### “免费代码”改变的是边际执行成本

一个有用类比是把 agent 看成多台可并行使用的施工设备。设备数量增加以后，搬砖速度显著提升；建筑图纸、承重规范、验收标准、安全边界和现场调度随之变得更关键。软件工程里对应的是任务定义、架构边界、测试、review、CI、文档和 guardrails。

第四个证据来自“一个 patch 约有 500 个小决策”。Ryan 指出，写好一份补丁依赖大量未明说的 **non-functional requirements**：可维护性、可靠性、可扩展性、类型边界、错误处理、可观测性、命名、代码布局、团队惯例、产品验收方式等。模型训练中见过大量代码，也见过这些要求的多种选择；如果团队没有把自己的选择写清楚，agent 就会在巨大可能空间中采样。

这解释了为什么“好工作”不能只靠一句任务描述。一个任务 ticket 往往只说明功能目标，却没有说明所有质量约束。Ryan 的主张是，团队要把这些 **non-functional requirements** 写成文档、ADR、persona-oriented documentation、历史 code review 经验、测试和规则，使 agent 能够看到“什么输出可以合并”。

## 1.5 风险与边界：免费代码仍会产生 slop

### code is free 的常见误读

**code is free** 不等于代码没有维护成本，也不等于可以无限堆积实现。代码仍会占用产品判断、架构边界、运行风险、测试维护、review 带宽和团队认知。Ryan 的结论恰好要求更强的 harness：代码越容易生成，越需要约束生成空间和验收标准。

本章最重要的风险词是 **slop**。它指低质量、重复、难以合并、需要人类反复纠偏的 agent 输出。Ryan 提到，团队不能只对 agent 说“不要产生 slop”然后期待问题消失；真正有效的做法是短期降低速度，回到任务和代码环境中观察 agent 为什么失败，再放置 **guardrails**，让相同错误下次被自动阻止或自动修复。

这形成一个重要边界：实现能力丰富会提高吞吐，也会提高错误传播速度。如果没有测试、review agents、lint、文档和验收标准，高并发 agent 可能同时制造更多合并冲突、更多风格漂移、更多隐性架构债。Ryan 的方法强调把 agent 的并行能力关进可观察、可纠偏、可接受的工程系统中；单纯追求更多 agent 会遗漏这层工程控制。

这里还需要保留对模型能力判断的归因。Ryan 在视频中主张模型已经能够承担完整软件工作，这来自他和团队的实践经验。写作和落地时应把它视为经验主张和组织假设，需要结合具体代码库、测试覆盖、任务类型、运行风险和团队验收能力验证。

## 从本章通向下一章

当代码实现不再是主要稀缺资源，工程师的高杠杆工作变成：写清目标、拆出任务、表达 **non-functional requirements**、建立 **guardrails**、安排 reviewer agents，并把每次人工纠偏沉淀为仓库中的可复用上下文。下一章将把这些机制拆成 harness 的基本构件。

## 1.6 本章小结

第一章回答的问题是：代码变便宜以后，工程师剩下的高杠杆工作是什么。Ryan 的答案是，工程师要从亲自执行实现，转向设计能让 agent 执行完整工作的系统。这个系统首先要承认新的稀缺资源：**human time**、**human and model attention**、**model context window**；然后把目标、标准、上下文、测试和反馈写进仓库，让 agent 在正确边界内消耗 token、生产候选实现、接受检查、修复问题并推进到可合并状态。

因此，**Harness Engineering** 是一种角色重构，远超一句工具口号：人类掌舵，agents 执行；人类定义方向和验收，agents 产出实现和修复；人类把反复出现的问题转化为长期 guardrails，agents 在这些约束中把实现能力变成团队可用的工程吞吐。

## 2 Harness 的基本构件：把“好工作”写成 agent 可读的系统

### 本章结论

Harness 的基本构件可以压缩为一个工程系统：**context delivery + guardrails + executable feedback loops**。团队把“什么算好工作”写进 **AGENTS.md**、persona-oriented documentation、**skills**、lint、**source-code tests**、错误消息、**reviewer agents** 和 **QA plan**，agent 执行时就能在正确时机看到标准、触发检查、收到可修复反馈，并把下一轮输出推向可接受代码。

Ryan 在这一段的核心判断是：高质量工程判断不能只停留在人的脑子里。前端架构、后端可扩展性、产品思维、安全可靠、QA 证明方式，这些经验原本通过低信号 code review、口头提醒和个人习惯传播；在 agent-first 工作方式下，它们需要变成仓库里可读、可调用、可检查、可复用的资产。一次写好的团队标准，会被每一条 agent trajectory 反复使用，这就是 Harness Engineering 的杠杆来源。

### 2.1 从个人经验到团队级上下文资产

“好工作”在真实软件工程里包含大量 non-functional requirements。单个 patch 需要处理可维护性、可靠性、可扩展性、产品验收、边界类型、失败恢复、日志和用户体验等细节。Ryan 的做法是让团队中每个 persona 把自己的判断写下来：前端架构师写组件与交互约束，后端工程师写扩展性和边界规则，产品型工程师写用户旅程与验收方式。

#### persona 文档的作用

persona-oriented documentation 的价值在于把一个人的隐性经验变成所有 agent 都能复用的显性上下文。它像团队里的“公共记忆”：某位工程师知道怎样写好的 **QA plan**，只要这套判断被文档化，每个后续 agent 任务就都能继承这项能力。



这类文档有两个直接后果。第一，人类可以减少通过重复 code review 传播同一条标准的同步成本。第二，agent 可以从这个仓库、这个产品、这个团队对“可合并代码”的具体定义出发，而非只依赖训练中见过的“平均代码风格”。Harness 因此承担了翻译层角色：把人的工程品味翻译成 agent 可执行的约束。

## 2.2 上下文会分页，所以约束必须持续刷新

长任务中的第一个现实问题是上下文会丢失。Ryan 提到 **auto-compaction** 已经持续改善，Codex 在长任务中能把对话压缩和分页；但工程上仍要假设 context 会随着时间被 paged out。换句话说，**AGENTS.md** 这类入口文件很重要，同时它只能承担一部分任务。agent 在执行过程中还需要被不断提醒：此处的网络代码要有 **timeouts and retries**，此处的接口要是 **secure interface**，此处的用户功能要有可验收证据。

### prompt injection 的语义边界

本章中的 prompt injection 指工程化上下文注入：把规则、提醒、审查意见和修复步骤送到 agent 当前轨迹中。安全语境里的恶意 prompt injection 是另一类风险，涉及攻击者诱导模型越权或忽略规则；两者需要分开讨论，避免把工程控制面和安全攻击混在一起。

Ryan 的解决方案是让 **reviewer agents** 在代码推进过程中不断检查 proposed patch。安全与可靠性 reviewer agents 可以在每次 CI push 上运行，读取相关文档和当前 diff，然后提出具体问题：网络调用是否带 **timeouts and retries**？新增 API 是否形成不容易误用的 **secure interface**？这类 reviewer agents 的角色类似长期在线的专家同事，只是它们把检查动作放进了自动化流程。

## 2.3 四类构件：文档、规则、检查器、执行工具

把 Harness 拆开看，可以得到四类构件。它们的共同点是：都在给 agent 提供上下文，都能影响 agent 的下一步行为。

| 构件  | 典型形态                                                               | 对 agent 的作用                             |
|-----|--------------------------------------------------------------------|-----------------------------------------|
| 文档  | <b>AGENTS.md</b> 、ADR、persona 文档、QA 指南                             | 说明这个仓库里什么算可接受工作，降低 agent 对隐性规范的猜测成本。    |
| 规则  | rules files、 <b>skills</b> 、团队约定、目录边界                              | 把任务入口、工具调用方式和局部约束写成可复用流程。               |
| 检查器 | bespoke lint、 <b>source-code tests</b> 、CI、 <b>reviewer agents</b> | 自动发现重复失败模式，把人工审查前移到执行轨迹中。               |
| 反馈  | lint error messages、test failures、review comments、PR 证明材料          | 用 <b>remediation steps</b> 告诉模型下一步如何修复。 |

表 1: Harness 的四类构件：它们分别负责说明标准、投递上下文、执行检查和提供可修复反馈。

Ryan 给出的第一个例子是 bespoke lint。团队可以写一个只服务于本仓库的 lint：每当代码调用 **fetch**，就检查它是否被 **retry** 和 **timeout** 包裹。这个规则看似很窄，实际价值很高，因为生产事故常常来自这类重复遗漏。写下规则之后，团队就把“可靠性 reviewer 是否记得提醒”变成了“检查器是否稳定运行”。

第二个例子是 **source-code tests**。如果 context window 是稀缺资源，就可以写测试限制源文件长度，例如文件不能超过 350 行。这种测试把检查对象从业务行为扩展到源代码结构本身。它让

仓库更适合 agent 阅读：较短文件、较清晰边界、更容易局部理解，也就更容易让模型把注意力放在当前任务上。

#### lint、test、error message 的共同点

lint、tests 和错误消息表面上属于质量工具，放进 Harness 视角后，它们都是 prompt delivery 介质。它们把团队标准转成模型能看见的文本，并在 agent 最需要修复方向的时候出现。

## 2.4 好的错误消息要能指导下一步修复

Ryan 特别强调错误消息的写法。一个普通 lint 只说“这里在循环里 await”或“这里出现 unknown 类型”，对 agent 的帮助有限；一个适合 Harness 的错误消息应该给出 **remediation steps**：为什么这里不该有 unknown，应该在边界处完成类型化解析，并且内部代码应使用由 **Zod** 推导出的类型。

这里的关键短语是 **parse, don't validate at the edge**。它表达的是一种类型边界纪律：外部输入进入系统时，边界层负责把不可靠数据解析成可靠类型；进入内部之后，业务代码面对的是已经被解析过的结构，因而无需在深层函数里到处写 **isRecord** 这类临时防御。对人类来说，这是类型系统和数据边界设计；对 agent 来说，它同时是一条能被错误消息触发的行动指令。

#### 错误消息也是工程接口

适合 agent 的错误消息需要包含三件事：失败事实、违反的团队原则、下一步修复路径。只报告失败会让 agent 继续猜；加入 **remediation steps**，错误消息就变成了一个小型 repair prompt。

## 2.5 Everything is a prompt：提示面分布在工程系统里

Ryan 用一句话概括这一段：**Everything is a prompt**。他展示的幻灯片写着 “You can just prompt things.” 这句话的重点是：团队可以在不改模型权重的情况下，通过各种介质改变模型在本仓库里的行为。

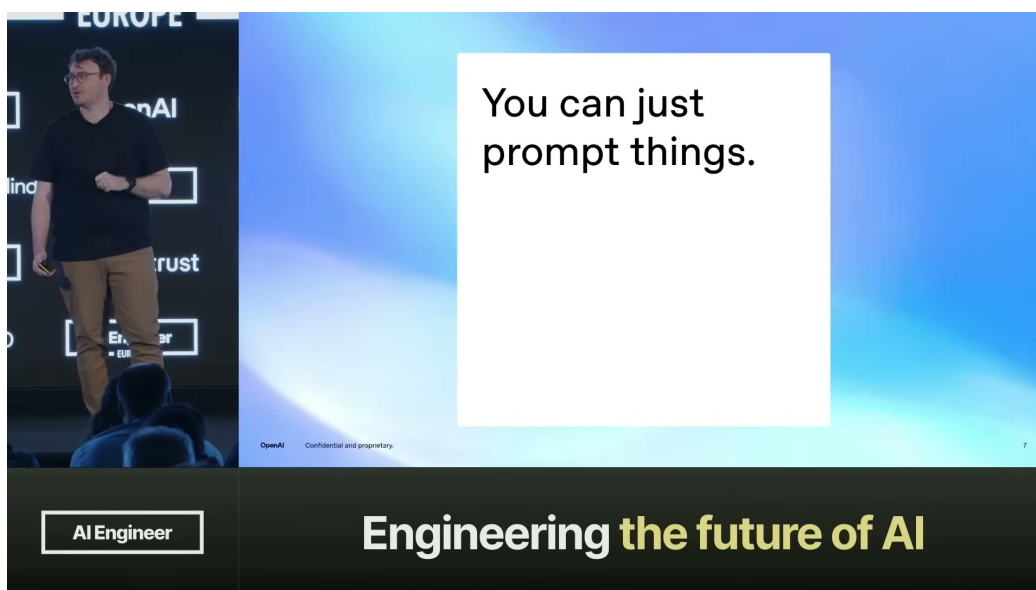


图 3: “You can just prompt things.”: Ryan 用一句口号强调提示可以嵌入工程系统的多个表面。<sup>3</sup>

这些 prompt surfaces 包括普通 prompts、rules files、**skills**、lint error messages、review comments、PR 上由 reviewer agents 注入的反馈，甚至还包括嵌入测试里的 agent SDK。只要某个文本会在 agent 执行、检查或修复时进入上下文，它就参与了 Harness 的控制面。

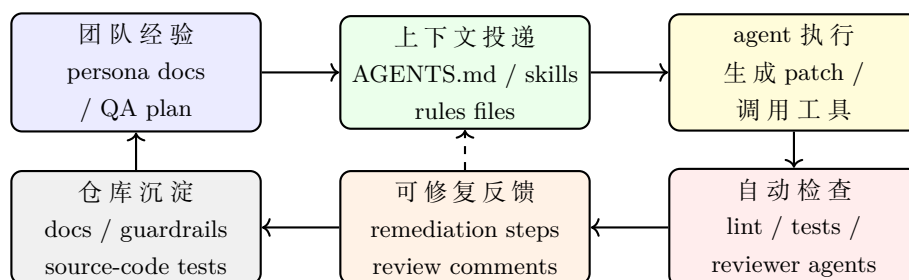


图 4: prompt surfaces 与 context delivery 的循环：团队标准通过不同介质进入 agent 轨迹，再通过检查和反馈回到仓库。

这里的结构可以理解为“把 prompt 放在 agent 需要它的地方”。任务开始时，**AGENTS.md** 和 skills 提供入口上下文；实现过程中，规则文件和工具说明提供局部做法；失败时，lint/test error messages 提供修复步骤；提交 PR 时，reviewer agents 和 review comments 继续注入验收标准。上下文因此从一次性说明变成持续投递。

## 2.6 QA plan：把产品验收也写进 Harness

本章最后回到产品型工程师的例子。一个好的 **QA plan** 需要列出功能、critical user journeys、用户如何使用 web app、API 和 service，以及 PR 里应该附上什么媒体证据来证明功能完成。这个计划写下来之后，review agent 就能检查每个 user-facing change 是否给出了足够证明：截图、录屏、日志、测试结果或其他验收媒体。

这件事的杠杆在于信任。人类看到 PR 中的证明材料，就更容易相信 agent 已经完成了正确工作；reviewer agents 也能基于同一套 QA plan 判断输出是否达标。Ryan 说这样会减少自己 shoulder surf agent 的需求，使自己从同步驾驶者的位置上移开，把更多工作委派给 agents。

### “The models crave tokens” 的工程含义

**The models crave tokens** 的边界需要说清楚：它强调模型需要足够的工具、tokens 和 context 才能完成 full job。Harness 的任务是把这些 tokens 组织成有用、及时、可验证的上下文，减少人类在每一步手动陪跑的同步负担。

因此，本章的 Harness 是一组彼此连接的工程接口：文档告诉 agent 什么重要，skills 告诉 agent 如何行动，lint 和 source-code tests 把结构问题变成失败信号，error messages 给出 remediation steps，reviewer agents 按 persona 检查非功能要求，QA plan 把产品验收写成可复用标准。每个接口都把“好工作”从隐性判断推进到可执行系统。

## 2.7 本章小结

本章回答的问题是：为什么写文档、lint、测试和错误消息是在“写给 agent 的工程接口”。答案是，这些 artifacts 都会在 agent 的执行轨迹中变成上下文和约束。人类过去通过经验、口头提醒

<sup>3</sup>视频画面时间区间：00:15:11–00:16:53。

和 code review 传递的判断，现在需要被写进仓库，成为 agent 能读取、能触发、能修复、能复用的 prompt surfaces。

Section 02 的可迁移结论有三条。第一，团队经验应文档化为 persona-oriented documentation 和 QA plan，让每条 agent trajectory 受益。第二，重复失败应转成 bespoke lint、source-code tests、reviewer agents 和 CI 检查，让 Harness 自动提醒 agent。第三，错误消息和 review comments 应包含 remediation steps，使反馈直接驱动下一次修复。把这些构件连接起来，Harness 就从“提示词集合”升级为 agent 可读、可执行、可验收的软件工程系统。

### 3 从 ticket 到 PR：Codex-first 工作流与协作面

本章的核心结论是：Ryan 团队的开发流程把 Codex 放在工程入口处，让 ticket、skills、本地工具、源码约束、GitHub PR 和 reviewer agents 共同组成协作面。这里的关键变化是入口从“人打开 IDE 后决定怎么做”迁移到“agent 接收工作、调用工具、产出 patch、接受反馈”。因此，一个好 harness 的设计对象应当是 agent 的调用路径：它在正确时间把正确文本、工具和反馈送到 Codex 面前。

Q&A 开始前，主持人重新介绍 Ryan 的背景：他刚完成 keynote，文章 *Harness Engineering* 与 *token billionaire* 的说法共同构成讨论背景。真正有教学价值的部分从实际 setup 问答开始：Ryan 没有展示笔记本，而是解释他们如何把工作从 ticket 推进到 PR。这个回答把前两章的抽象概念落到一个具体工程流中：人类负责定义 work item 和接受标准，Codex 负责在 harness 中执行，PR 负责把所有反馈汇聚成可协作的公共表面。

#### 3.1 从 ticket 开始：把工作切成 agent 可接收的任务

Ryan 描述的起点是 tickets：团队把 features、reliability work 和其他 chunks of work 写成可交付单元，再把 ticket 交给 agent。这个设计有一个朴素前提：agent 要想独立完成任务，必须先拿到足够明确的工作边界。ticket 像物流单，写清楚目的地、包裹内容和签收标准；Codex 像执行者，依据这张单据进入仓库、调用工具、提交结果。

##### Q&A：实际工作流的入口 (00:20:06–00:21:16)

主持人：What’s your workflow like? How do you approach a task?

Ryan：We start with tickets. We give that ticket to an agent along with a couple of skills.

Codex is the entry point. We do things outside in.

这段问答的重要性在于它给出了 Codex-first 的工程边界。所谓 **Codex is the entry point**，含义是仓库里的本地开发体验、调试工具和约束检查都应能被 Codex 首先调用。所谓 **outside in**，含义是团队先承认 Codex 作为外部执行者进入系统，再围绕它暴露工具和说明；它像一位新同事从门口进入办公室，标识、钥匙、流程单和监控屏都要放在入口附近，无需假设他已经坐在某个熟悉 IDE 里。

##### Codex-first 的最低工作单元

一个 Codex-first ticket 至少包含三类信息：要完成的用户或可靠性目标、Codex 可调用的 skills，以及完成后要通过的本地检查或 PR 反馈。ticket 本身保持克制，负责指向能按时暴露知识的 harness。



### 3.2 Codex 调用链：skills 把本地工具变成 agent 接口

在 Ryan 的 setup 中，**skills** 的角色超出装饰性 prompt：它是 Codex 操作本地系统的入口。一个 skill 教 Codex 启动 app；另一个 skill 启动 **local observability stack**，让日志和 telemetry 可见；还有一个 skill 通过本地 CLI 和 daemon 接入 **Chrome DevTools**。这些工具共同回答一个实践问题：agent 改完代码后，怎样观察运行状态、调试浏览器、确认行为符合预期。

这一点也解释了为什么 Ryan 不主张无限扩张 skills 数量。团队把 leverage 集中在大约 5–10 个高价值 skills 上，因为仓库基础设施和本地开发工具变化很快。技能面过宽会制造维护负担；核心 skills 变强之后，人类只需调用稳定入口，Codex 可以根据文档和工具变化继续完成工作。Ryan 提到他们从直接使用 Chrome DevTools protocol 迁移到 daemon 方案后，他大约三周都没有意识到变化，因为 Codex 已经能通过文档和 skill 继续完成同类任务。

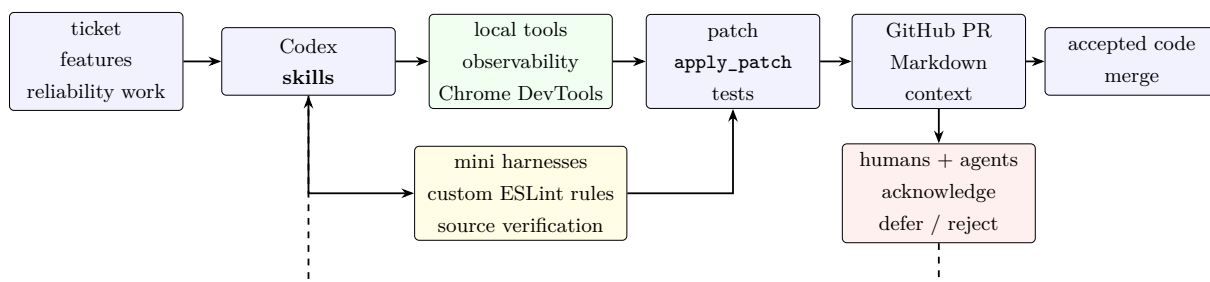


图 5: Codex-first 工作流：ticket 进入 Codex，skills 暴露本地工具，guardrails 修正 patch，PR 汇聚人类与 agent 反馈。图根据 00:20:30–00:29:59 的 Q&A 内容重绘。

这张图把本章的协作面压缩成一条链：ticket 只负责启动任务，Codex 通过 skills 进入本地系统，工具和 guardrails 让 patch 可观察、可验证，GitHub PR 让反馈变成共享广播域。真正的工程设计点是这些工具能否被 Codex 按需要调用。

### 3.3 本地 guardrails：custom ESLint rules 与源码结构验证

Codex-first 仍然需要明确约束。Ryan 团队在仓库内放了许多 mini harnesses，方便继续追加 guardrails。典型例子是全 workspace 接入的一大包 **custom ESLint rules**，以及更高层的源码结构验证。后者检查代码本身的组织方式，关注点包括 **package privacy**、不同 stack layer 之间的 **dependency edges**、跨文件 Zod schema 去重、以及 async helpers 是否使用单一 canonical implementation。

这些规则针对的是 agent 的一个常见行为：它有时会优化局部 package 的一致性，忽略团队已经沉淀的 shared utilities。局部看起来合理的 patch，可能会复制 schema、绕过共享 helper、破坏 package 边界。source verification 的价值就在这里：它把“团队希望怎么写”变成可执行检查，让人类 reviewer 少花注意力在重复、低价值的纠偏上。

#### source verification 与普通测试的互补关系

普通测试主要回答“运行行为是否正确”，source verification 回答“代码结构是否符合团队约束”。前者像验收产品功能，后者像检查工厂生产线的零件规格。两者互补：行为正确的代码仍可能破坏长期可维护性；结构正确的代码仍需要功能测试证明用户路径成立。

这里还隐藏着一个工程依赖：所有检查必须有清晰的 remediation path。若 lint 只说“失败”，Codex 只能猜测修复方式；若错误消息说明应使用哪个 helper、哪个 package 边界不可跨越、哪类



schema 已有 canonical 版本，它就成为下一步修复的 prompt。Section 02 已经说明 “Everything is a prompt”，本章补上实际落点：prompt 可以出现在 ESLint、source-code tests、review agents 和 PR 评论里。

### 3.4 最低充分上下文：在正确时间给模型正确文本

Ryan 对“过度设计 harness”的回答可以概括为一句话：一个好 harness 只做最低充分的 context management，把正确文本在正确时间交给模型。这个判断来自第一性原理：模型必须知道任务要求、guardrails 和完成标准；同时，模型的 context window 与 attention 有限，前置灌入所有说明会制造噪声。

因此，Ryan 倾向于 just-in-time context。以 React UI 为例，初始阶段可以让 agent 先探索、prototype 和 cook；到了 lint 或 test 阶段，harness 再提示它拆分 component、让局部单元更 stateless、为 snapshot tests 保留稳定边界、避免不合适的 prompt drilling。agent 得到新约束后，会基于已经写出的 patch 做二次整理，然后推送到 GitHub。

#### 好 harness 的核心职责

好的 harness 负责把 **right text, right context, right time** 送到 agent 面前。它让上下文随着任务阶段逐步出现：启动时给目标，开发时给工具，检查时给约束，PR 时给反馈，避免把全部知识一次性灌入 Codex。

这个原则也说明为什么 context 不会随着模型能力提升而消失。模型能力增强可以提高执行质量，却不会自动知道某个团队此刻的产品目标、package 边界、review 标准和 release 风险。harness 的稳定价值在于把这些局部知识变成可传递、可延迟、可执行的上下文。

### 3.5 first-party harnesses: post-training 带来的工具语义杠杆

当主持人追问 Codex harness 与其他 harness 的比较时，Ryan 给出的是一个有边界的判断：实验室不仅会做模型的 **post-training**，也可能在模型主要部署的 harness 语境中训练模型。因此，**first-party harnesses** 可能带来额外杠杆，例如模型熟悉 `apply_patch` 这类工具、熟悉 `bash` 调用的 quoting semantics、熟悉它在训练循环中反复看到的工具协议。

#### first-party harnesses 的合理解

Ryan 的观点可以理解为“利用模型已经熟悉的工具语境”。这是一条有边界的实践判断：团队可以优先把精力放在“什么是正确代码”上，把 coding harness 的底层细节交给 Codex、Claude Code 等一线工具团队持续改进。

这会改变团队自建工具的边界。若目标是复制整个 coding harness，团队要持续追踪工具协议、shell 语义、patch 应用、模型行为差异等底层细节；若目标是通过 SDK 或 Codex app server 接入 first-party harnesses，团队可以把注意力上移到 steering：观察不同模型版本的行为差异，设计 repo docs、skills、lint、tests 和 reviewer agents 去塑造可接受输出。

### 3.6 PR 作为 hub-and-spoke broadcast domain

Ryan 对协作平台的回答非常务实：主要使用仓库里的 Markdown 文件和 GitHub。Markdown 承载持久说明，GitHub PR 承载具体 diff、讨论和验收反馈。他把 PR 类比为 Google Docs 的协作

文档：大家围绕同一个 work artifact 提建议、回复评论、应用修改。不同之处在于这里的参与者同时包含 humans 和 agents。

因此，PR 成为 **hub-and-spoke broadcast domain**。hub 是 PR 本身，spokes 是实现 agent、reviewer agents、人类 reviewer、CI 和仓库文档。团队优化的是 throughput，所以任何参与者都可以 review 或暂时不 review；实现 agent 可以对反馈做 **acknowledge/defer/reject**，也就是承认并处理、延后处理、或基于交付判断拒绝处理。

### 过度约束 reviewer 反馈的风险

若规则要求实现 agent 必须处理每一条 reviewer comment，agent 可能被所有 reviewer comments “bully”。Ryan 的交付目标偏向 accepted, valuable code：代码应当能推进产品和仓库状态，避免被 minutia 淹没。自动 reviewer 的职责是浮出真正影响 merge 的问题，避免制造无限待办列表。

这一点把协作面和 guardrails 重新连在一起。reviewer agents 的价值在于让关键约束在 PR 上可见，给 Codex 提供可判断的反馈。实现 agent 仍要做判断：哪些反馈阻塞交付，哪些反馈属于后续工作，哪些反馈与任务目标不符。人类也要接受一个现实：高吞吐协作需要明确优先级，PR 是广播域，也是取舍场。

## 3.7 本章小结

本章回答了“为什么 harness 要围绕 agent 的调用路径设计”这个问题。原因很直接：在 Codex-first 工作流中，真正执行任务的是 Codex，入口、工具、上下文、反馈和验收都必须能沿着它的路径抵达。若仓库仍只围绕人类 IDE 习惯设计，agent 会缺少启动 app、观察日志、接入 Chrome DevTools、理解 package privacy、处理 dependency edges、响应 PR 反馈的稳定接口。

可以把 Section 03 的机制压缩成四句话：

- ticket 定义工作边界，Codex 通过 **skills** 接收可操作入口；
- **local observability stack**、**Chrome DevTools** 和本地 CLI 让 agent 能观察真实运行状态；
- **custom ESLint rules** 与 source verification 把 package 边界、共享实现和 dependency rules 写成可执行 guardrails；
- GitHub PR 作为 **hub-and-spoke broadcast domain**，让 humans 与 agents 围绕同一个 diff 做 **acknowledge/defer/reject** 式协作。

由此可见，Codex-first 的本质是把软件开发流程重排为“工作定义、工具暴露、执行修正、PR 协作、反馈沉淀”的 agent 可执行系统。下一章将把这个系统继续扩展到仓库结构本身：当代码、目录和 package 边界也成为 prompt 时，代码库就开始变成一台上下文机器。

## 4 把仓库改造成上下文机器：结构、一致性与 progressive disclosure

本章的核心结论是：从手写代码迁移到 agent-first 工作方式，第一步应落在把现有仓库改造成一个更容易被理解、验证和局部修改的上下文机器。Ryan 在这段 Q&A 中把迁移路径分成两层：个人先用 agent 增加 **more tests** 与 **confidence in code**；团队再通过包边界、一致性工具和 **progressive disclosure** 降低 agent 的搜索成本。

所谓“上下文机器”，可以理解为一座标注清楚的图书馆。书本本身仍然是代码，目录、索引、借阅规则和分类法则对应测试、包边界、lint、CI、工具约定和文档。读者换成 coding agent 后，图书馆的组织方式会直接影响它能否在有限 *context window* 内找到正确信息，并产出可预测的修改。

### 本章主张

代码库结构本身是 agent 的上下文接口。目录边界、公共 API、测试、共享工具、CI 脚本和 lint 规则都在向 agent 说明：哪里能改、怎样改、如何证明修改已经完成。仓库越一致，agent 产出的 token 越容易被预测、审查和接受。

## 4.1 个人迁移路径：先补测试，再审计时间

Ryan 给出的起步建议从两个实际问题出发。第一个问题是“代码是否足够可信”。对手写代码较多的团队来说，让 agent 先补充测试通常比直接交付大功能更稳健。agent 擅长阅读现有代码，再结合使用方式写出行为断言；这些测试把隐含需求转成可执行证据，使团队获得更高的 **confidence in code**。这也会反过来提升 agent 后续导航仓库的能力，因为它可以通过测试知道哪些行为被保护。

这里的 **more tests** 指向有效行为约束，数量本身不是目标。测试的价值在于把“程序应当如何响应用户交互”写成可运行约束。若测试只覆盖实现细节，agent 仍然可能在重构时满足测试却破坏产品意图；若测试覆盖关键行为、边界条件和常见失败路径，它就成为 agent 的地图与护栏。

第二个问题是“人的时间消耗在哪里”。Ryan 建议工程师观察自己一天中被什么阻塞：在编辑器中手写代码、等待测试运行、等待 human review、等待慢 CI、处理 flaky tests，还是重复解释同一种实现约定。适合交给 agent 的起点通常出现在这些等待与重复中。agent 可以增量自动化瓶颈，使工程师从亲自执行转向 **sequencing and orchestration**：定义任务、排序任务、安排执行、提供团队可以复用的构件。

### 从执行者到 sequencing and orchestration

**sequencing and orchestration** 的工程含义是：工程师的主要杠杆从“亲自完成每个实现细节”转向“让正确任务以正确顺序被正确执行”。这类似技术负责人先定义 Kafka consumer 的标准构件，再让团队成员按同一模式接入事件，从而避免每次都从头审查 consumer 是否正确处理重试、幂等和观测。

## 4.2 无人值守任务：tests are green 是最低完成信号

Ryan 的“car workflow”例子把 harness 的目标讲得很具体：他离开办公室前启动任务，把电脑连到手机网络，让 agent 在通勤时间继续运行。这个例子的关键是任务说明和 skills 已经足够清楚，agent 知道自己要持续推进，直到 **tests are green**。

### 高信息问答片段 (00:32:29–00:33:38)

**主持人**：你如何在车里和 agents 一起工作？

**Ryan**：离开办公室前启动任务，skills 会告诉 agent：go until the **tests are green**。

**Ryan**：每次人类需要输入 continue，都说明 harness 没有给出足够上下文来定义“继续直到完成”。

这个例子揭示了长任务的隐藏依赖。agent 需要知道完成条件、允许采取的下一步、失败后如何恢复、测试失败时是否应继续修复、何时停止并请求人类判断。人工反复点击 `continue` 暴露的是完成语义没有写进 `harness`。若团队希望 agent 长时间运行，完成标准必须被写成测试、命令、日志、验收规则和失败恢复路径。

### 4.3 progressive disclosure: 上下文按时机释放

当组织知识图谱变大后，把所有文档一次性塞给 agent 会制造噪声。Ryan 在 ‘00:33:38–00:34:00’ 的问题中回应 **progressive disclosure**：更大的代码库、更大的团队和更多的领域知识，需要按任务阶段逐步释放上下文。它与 *context window* 是一对互补概念：*context window* 是容量限制，*progressive disclosure* 是容量管理策略。

#### progressive disclosure 与 context window

**progressive disclosure** 的目标是让 agent 在每个阶段看到足够相关的信息。任务刚开始时需要目标、边界和验收条件；实现时需要局部目录、共享工具和示例；验证时需要测试、`lint`、`review rules` 和失败修复步骤。上下文释放过早会稀释注意力，释放过晚会导致 agent 猜测团队规则。

仓库结构正是 *progressive disclosure* 的物理载体。目录树、包边界和公共接口会告诉 agent：当前任务属于哪个局部子树，哪些 API 是公共入口，哪些实现细节应留在内部，哪些共享工具必须复用。文件系统中的 `code` 也是 `text`，因此代码本身也在持续向 `coding agent` 发出 `prompt`。

### 4.4 从单包混乱到 PNPM workspace: 让边界可见

Ryan 回顾了一个从 `blank repository` 和 `create electron app` 起步的项目。最初的单包结构很快变成混乱状态，原因包括缺乏 **package privacy**、无法强制 `public API` 与 `internal API` 的边界、文件系统中没有清楚的 `hooks` 来表明不同 `domain` 之间的隔离。后来团队采用非常重的架构拆分：**750 packages in the PNPM workspace**，按 **business logic domain** 或 `stack layer` 隔离，并把可复用能力封装进小 `util packages`。

#### 不要把 750 packages 当成通用处方

把 Ryan 的 **750 packages** 当成所有团队都应模仿的目标，是错误假设。这里的教学重点局部可理解、边界可验证、上下文可迁移。小团队、单体应用或早期产品可能只需要清晰目录、少量包边界和统一工具；盲目追求包数量会增加构建、版本、依赖和维护成本。

这段经验的可迁移原则是：即使系统没有微服务，也可以把仓库组织成多数修改都能落在一个局部子树内。agent 在局部子树内工作时，搜索空间更小，依赖关系更明确，测试和 `lint` 更容易定位。对人类而言，这是模块化；对 agent 而言，这是可检索、可验证、可迁移的上下文边界。

| 仓库结构措施                                          | 给 agent 的上下文信号                      | 工程收益                                     |
|-------------------------------------------------|-------------------------------------|------------------------------------------|
| 行为测试与 source-code tests                         | 当前行为、结构规则和完成条件被写成可执行约束              | 增加 <b>confidence in code</b> ，减少纯人工复查压力  |
| <b>package privacy</b>                          | public API 与 internal API 的边界可被工具强制 | 降低误用内部实现、跨域耦合和 accidental dependency 的概率 |
| <b>PNPM workspace</b> 与局部 package               | 任务通常可以限制在某个 package 或目录子树           | 减少搜索范围，支持 <b>progressive disclosure</b>  |
| 按 <b>business logic domain</b> 或 stack layer 隔离 | 领域边界和技术层级在文件系统中可见                   | agent 更容易判断应修改业务逻辑、基础设施层还是共享工具           |
| 小 util packages 与强制复用规则                         | 常见能力有唯一入口，并可通过 lint 要求使用            | 把团队经验编码成可复用 leverage                     |
| 统一 helper、ORM、CI、lint 添加方式                      | “正确做法”在仓库不同位置保持同形                   | 提高 token 输出的一致性和可预测性                     |

表 2: 把仓库结构转化为 agent 可用上下文的方式（对应 00:34:00–00:36:20）。

#### 4.5 make things the same: 一致性降低注意力成本

Ryan 在这一段反复强调 **make things the same**。一致性属于注意力管理策略，远高于审美偏好。agent 在仓库不同位置看到相同模式时，可以把一个局部学到的上下文迁移到另一个局部；当每个子系统都有不同 ORM、不同 CI 脚本、不同 lint 扩展方式和不同副作用命令封装时，agent 必须反复重新理解环境。

因此，Ryan 提出一组非常具体的“一种方式”：

- 一种 **bounded concurrency helper**，用来表达有限并发的标准实现。
- 一种 **observable and instrumented side effectful command**，用来封装有副作用、需要观测和埋点的命令。
- 一种 ORM，避免数据访问模式在仓库中分裂。
- 一种编程语言，或至少一个清晰的语言边界策略。
- 一种 CI script 写法，使构建、测试、发布步骤可迁移。
- 一种添加 lint rule 的方式，使新 guardrail 能自然进入仓库。

这些统一点共同服务于一个目标：让模型需要产出的 token 更容易预测。换句话说，仓库越像一组重复出现的语法和范式，agent 越容易在局部上下文中补全团队希望看到的代码形状。这里的“一致”并不要求所有业务都被压成同一个抽象；它要求相同类别的问题使用相同类别的解决方式。

##### 一致性的判据

有用的一致性应当降低搜索、解释和验证成本。若某个统一规则能让 agent 更快找到示例、更少发明新模式、更容易通过测试和 lint，它就属于 harness 的资产；若某个统一规则只增加 ceremony，难以减少错误或审查负担，它应被重新评估。

#### 4.6 隐含依赖与风险边界

本章方法依赖三个前提。第一，测试和 lint 需要表达真实产品行为与工程约束，表面覆盖率无法替代有效断言。第二，包边界需要工具强制，单靠文档说明很容易在高速度 agent 开发中漂移。第



三，progressive disclosure 需要稳定入口，例如 AGENTS.md、skills、目录 README、错误消息和 reviewer agents；若入口分散，agent 仍会在上下文中迷路。

主要风险也很清楚。过度拆包会制造构建复杂度；过度统一会压制合理的领域差异；过早架构化会让早期产品承担组织级成本。正确做法是从重复错误、反复 review comment、CI 等待和跨域耦合这些可观察痛点出发，再把解决方案沉淀成测试、包边界和统一工具。

## 4.7 本章小结

Section 04 把 Harness Engineering 从工具使用推进到仓库设计。个人层面的起点是让 agent 写 **more tests**，提升 **confidence in code**，再审计时间瓶颈，把等待、重复和机械检查逐步自动化。团队层面的重点是让工程师进入 **sequencing and orchestration** 角色：定义任务顺序、完成标准和可复用构件，使 agent 能在更少人工继续指令下运行到 **tests are green**。

组织级仓库需要通过 **progressive disclosure** 管理上下文，通过 **package privacy**、**PNPM workspace**、**business logic domain**、小 util packages 和一致的工具入口降低搜索成本。最终原则可以压缩为 Ryan 的一句话：**make things the same**。仓库一致性越高，agent 从一个局部迁移到另一个局部时越少重新学习，产出的 token 越可预测，验证和接受成本也越低。

# 5 反馈闭环：从 *code review* 到自动 *guardrails*

## 5.1 结论：把人工评审转成长期约束

Ryan Lopopolo 在这一段回答的核心结论是：高速度 agent 开发会先把协作成本放大，尤其是 **code review**、**merge conflicts** 和 PR 等待时间；真正可持续的做法，是把评审中反复出现的 **slop** 视作 **context failure**，再把它沉淀为仓库文档、规则、测试和 **reviewer agent**。这样，一次人工评审既能解决当前 PR，也会变成以后自动阻止同类问题的 **guardrail**。

这段内容承接上一章的“代码库结构本身是 agent 的上下文接口”。如果仓库已经尽量局部化、一致化，下一步就是让反馈也局部化、一致化、可复用。换句话说，团队需要同时提高 PR 产量，以及“问题被分类、被记录、被自动拦截”的速度。

### 重复 review comment 是 harness 缺口候选

如果同一种 **code review** 评论在一周内反复出现，它应被当作 harness-gap candidate：agent 没有在正确时间看到正确上下文，或者没有被测试、lint、规则文件、文档、**reviewer agent** 及时约束。人工评论只是表层症状；可复用的修复是把这类评论转写成仓库中的自动反馈机制。

## 5.2 动机：高 PR 速度会放大同步成本

主持人的问题很直接：当团队有很高 velocity 时，还要不要人工读代码？能否只信任测试覆盖？如何越过“每个 PR 合并前都必须手动检查一遍”的心理门槛？Ryan 的回答避开“读”或“不读”的简化二分，回到上一章已经出现的方法：观察团队时间花在哪里，然后系统性减少同步阻塞。

Ryan 给出的具体瓶颈是 PR 速度。每个工程师每天可以产出 3 到 5 个 PR；即使只有 3 人团队，也会很快遇到 **merge conflicts**。原因并不复杂：agent 生成的 PR 往往偏大，多个工程师和多个 agent 又可能同时改同一片代码。此时，冲突已经成为高吞吐系统中的拥塞信号。

Ryan 提到两条缓解路径：

- **tree out the code**: 把代码结构拆得更清楚, 让常见改动尽量落在不同子树、包或边界内, 从源头减少 **merge conflicts**。
- **缩短 PR open time**: 减少 PR 挂起等待的时间, 降低其他改动插入同一代码区域的概率。

这两条路径指向同一个系统目标: 减少同一资源上的排队。代码结构拆分是在空间上减少竞争; 缩短 PR 开放时间是在时间上减少竞争。问题随即推进到真正的阻塞点: PR 之所以挂得久, 是因为需要 **code review**, 而 **human reviewers** 成了同步瓶颈。

### 5.3 *garbage collection day*: 把一周的 *slop* 分类

Ryan 团队采用的实践叫 **garbage collection day**: 每周五, 团队把一天用于收集本周观察到的所有 **slop**, 尤其是那些让 PR 难以合并的问题, 然后寻找“如何从类别上消灭它”的办法。这里的关键动作是把坏补丁背后的原因归类; 单次坏补丁清理只能解决当前症状。

这和编程语言里的垃圾回收有一个直观类比: 程序运行时会产生不断产生临时对象, 垃圾回收器周期性回收无法继续使用的对象; **agent-first** 团队会不断产生候选实现, **garbage collection day** 周期性回收评审中暴露的低质量模式。软件团队回收的对象是重复的评审负担、模糊的团队惯例和遗漏的上下文。

#### *slop* 的分类方式

**slop** 可以按产生原因分类: 缺少架构边界、缺少可靠性标准、缺少前端一致性规则、缺少可扩展性约束、缺少测试模式、缺少错误处理惯例。也可以按 **reviewer persona** 分类: **front-end architect**、**reliability engineer**、**scalability reviewer**。分类越稳定, 越容易把人工经验转成自动检查。

Ryan 对这一机制的解释是: 人类在 PR 上给出的反馈, 说明 **agent** 在某个地方发生了 **context failure**。它可能不知道“好代码”在这个仓库里是什么意思, 也可能没有看到某个设计文档, 或者没有被现有测试提醒。于是, 正确动作是把反馈放回仓库, 让后续 **agent** 在写代码、跑测试、接收 **review** 时自动遇到这条约束。

### 5.4 机制: 从评论到文档、测试和 **reviewer agents**

这条路径可以概括为一个反馈闭环: 观察低质量输出, 归类原因, 沉淀到 **docs/rules/tests**, 交给 **reviewer agents** 和 PR 反馈面执行, 再观察 **slop** 是否减少。下面的图把这一段 Q&A 的机制重绘成可复用模板。

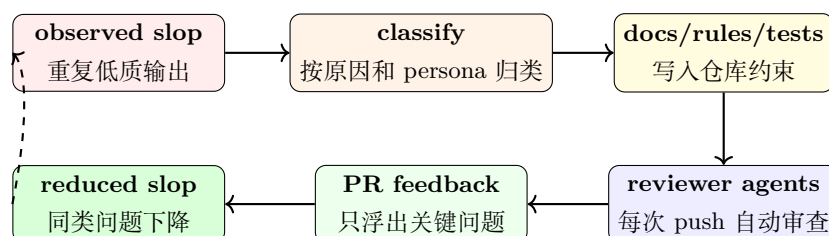


图 6: 从 *observed slop* 到 *reduced slop* 的自动反馈闭环。<sup>4</sup>

闭环中最重要的转换发生在中间三步:

<sup>4</sup>机制来源时间区间: 00:36:46–00:39:28。该图为基于字幕内容重绘的教学图。

- **从 PR 评论到文档**：把“这个接口不应该这样暴露”“这个组件不符合前端架构”“这里缺少 timeout/retry”等评论，转成可被所有 agent 读取的规范。
- **从文档到 tests/rules**：把可机械判断的问题写成测试、lint、source-code tests 或 CI 检查，让 agent 在提交前或提交后收到明确反馈。
- **从文档到 reviewer agents**：把需要语义判断的问题交给按 persona 运行的 reviewer agents，让它们用同一批文档判断 PR 是否达到团队标准。

Ryan 的表述里有一个很关键的动作：把这些 docs 放在一个所有流程都能 attach 的位置。这样，coding agent、测试、lint、CI、PR reviewer 和 **reviewer agent** 看到的是同一个标准源。标准分散时，团队会得到互相冲突的反馈；标准集中时，反馈才可能被持续追加、持续执行、持续减少噪声。

## 5.5 Persona reviewer：只浮出能阻塞合并的问题

Ryan 团队把 review feedback 按 persona 分桶，例如 front-end architect、reliability engineer、scalability reviewer。每个 persona 都代表一组团队质量标准：前端架构关注组件边界和交互一致性；可靠性关注重试、超时、错误处理和可观测性；可扩展性关注依赖方向、数据访问路径和长期演进空间。

然后团队为每个 persona 启动一个 **reviewer agent**，在每次 push 时审查代码。它的提示目标应聚焦高影响问题：检查这段代码是否足够好，基于描述“好工作”的文档，只浮出 **P2s or above**，也就是那些会 **block this PR from merging** 的问题。

### 过度 noisy 的 reviewer agents 会制造新瓶颈

自动 reviewer 如果输出大量 minutia，会把 human review 的同步瓶颈搬到 agent review 上。有效的 **reviewer agent** 应聚焦足够重要的问题，例如 **P2s or above**、会 **block this PR from merging** 的风险、违反核心架构或可靠性约束的缺陷。低优先级风格建议应被降噪、延后或转成格式化工具和 lint 的机械反馈。

这里有一个重要边界：persona reviewer 的职责是承担可复用预审，把显然不该进入合并队列的问题提前拦住，把重复评论自动化，把人类 reviewer 的注意力释放给产品判断、架构取舍和高风险变更。对团队来说，这是一种注意力分层：机器处理可分类、可复现、可规则化的反馈；人类处理需要上下文权衡和责任承担的反馈。

## 5.6 可迁移模板：从一次评审到长期 guardrail

把 Ryan 的实践迁移到其他团队，可以按五步执行。

1. **记录重复评论**：每次 **code review** 中出现相同意见时，记录评论文本、触发代码、影响范围和合并阻塞程度。
2. **判断是否为 context failure**：检查 agent 是否缺少相关文档、示例、测试、规则、错误消息或 persona 视角。
3. **选择承载面**：机械问题进入 lint/tests/source-code tests；语义问题进入 docs 和 **reviewer agent**；流程问题进入 PR 模板、CI 或 task instructions。
4. **接入 PR 路径**：让新 guardrail 出现在 agent 写代码后、PR 打开前、每次 push 后，或者 reviewer agents 执行时。

5. **观察** **slop reduce, reduce, reduce**: 每周复盘同类问题是否下降; 如果仍反复出现, 说明 **guardrail** 放置位置、触发条件或文档表达仍不够有效。

这套模板的底层假设是: 团队的“好工作”标准足够稳定, 值得沉淀成自动机制。如果某个反馈仍处在探索阶段, 或者高度依赖产品策略和一次性判断, 它可能暂时更适合由人类保留; 过早写成阻塞规则会放大错误标准。自动化越强, 错误规则的放大效应越高。

## 5.7 本章小结

本章回答的问题是: 如何把一次人工 **code review** 变成长期自动 **guardrail**。Ryan 的路径是先承认高速度 **agent** 开发会制造新的协作压力: 3 到 5 个 PR 每人每天会放大 **merge conflicts**, PR open time 会放大排队, 人类 review 会成为同步瓶颈。然后, 团队用 **garbage collection day** 收集一周内的 **slop**, 把重复评论解释为 **context failure**, 再转写成仓库文档、规则、测试和 **reviewer agents**。

这套闭环的目标是让同一种错误越来越难重复出现。每条高频评审意见都可以成为 **harness** 缺口候选; 每个缺口都应被放进合适的承载面; 每个自动 **reviewer** 都应只浮出足够重要、可能 **block this PR from merging** 的问题。最终, 团队看到的结果应是 Ryan 所说的 **slop reduce, reduce, reduce**: 低质量输出持续下降, 人类注意力从重复评论转移到真正需要判断的工程问题上。

## 6 总结与延伸: *LLM as fuzzy compiler* 与长期 autonomous engineering

### 6.1 结论: 未来的软件工程输入会更像约束、指标和验收标准

Ryan Lopopolo 在最后一段 Q&A 中把 **Harness Engineering** 推到更高层: 当代码实现的边际执行成本下降以后, 工程师最重要的资产逐渐转向任务排序、上下文组织、**constraints**、**optimization passes**、**acceptance criteria**、**success metrics** 和 **reliability metrics**。代码仍然需要存在、运行和被维护, 但它在这套图景中更接近一个由规格、上下文和验证流程编译出来的候选产物。Ryan 用 **LLM as fuzzy compiler** 概括这种心智模型: LLM 接收人类写下的目标、仓库上下文和工程约束, 生成候选代码; 测试、**reviewer agents**、CI、QA 和人类验收再决定这些候选代码是否成为被接受的工程事实。

这也是全文的最终综合: **humans steer, agents execute** 的关键在于把软件工程改造成一套可被 **agent** 持续执行、持续检查、持续修复的系统。前五章讨论的 **code is free**、文档、skills、Codex-first workflow、progressive disclosure、code review 闭环, 在终章被压缩为一个更抽象的判断: 工程师正在为 **agent** 编写一层“工程过程程序”, 这就是 Ryan 所说的 **meta programming**。

### 6.2 Token 预算: 大致三分, 避免精确会计

主持人追问 Ryan 的十亿级 token 消耗主要花在哪里。Ryan 的回答很谨慎: 大致可以看成 **roughly a third/third/third**, 分布在 **planning**、**ticket curation**、**documentation**、**implementation**, 以及 CI 中运行的东西。这里的重点在于 token 已经变成需要管理的工程预算。一个团队需要知道 token 被花在定义工作、生成候选实现、验证和修复上, 同时避免只盯着“写了多少代码”。

| Token 去向                               | Ryan 的量级说法 | 工程含义                                                                                   |
|----------------------------------------|------------|----------------------------------------------------------------------------------------|
| planning、ticket curation、documentation | 约三分之一      | 把目标、里程碑、上下文、 <b>acceptance criteria</b> 和团队标准写成 agent 可执行的输入。                          |
| implementation                         | 约三分之一      | 让 agent 生成、重构、删除和修复代码；这部分体现 <b>code is free</b> 的执行容量。                                 |
| CI、review、verification                 | 约三分之一      | 让 tokens 消耗在测试、reviewer agents、source checks 和可接受性判断上，使 written code 变成 accepted code。 |

表 3: Ryan 对 token 使用的粗粒度描述：约三类支出各占一部分，不能理解成精确统计。<sup>5</sup>

这个表格也解释了为什么 Ryan 认为 CI token 是必要支出。写出代码本身的难度已经下降；让代码被接受、推进代码库和产品向前，才让 written code 产生价值。换言之，token 花在 CI、review 和验证上，是把候选实现转化为团队可合并产物的成本。

### 6.3 *plan mode* 与 *exact plans*：计划本身也是代码路径

Ryan 谈到 **plan mode** 时没有把它视为当前 harness 的核心入口。他提到自己使用过 **exact plans**，那是一种较早发布的 proto skill，用来规定计划应怎样包含 milestones 和 **acceptance criteria**。他的期望更激进：理想情况下，工程师可以丢入一个 ticket，agent 就能在既有 harness 中完成工作，而不必每次都绕过一个单独计划阶段。

这条建议保留计划的价值，同时要求计划承担与代码相近的审查强度。Ryan 真正强调的是：计划一旦被批准，就会变成 rollout 的实际指令。如果人类批准了一份没有认真阅读的计划，等于把一串未经审查的约束写进 agent 的执行轨迹。对高风险工作，Ryan 建议把 plan 单独作为 PR 提交，让人类逐行 review，并在批准后再启动执行。

#### 未阅读的计划会变成坏指令

**plan mode** 和 **exact plans** 的风险在于：计划批准动作会把计划内容编码为后续执行约束。若计划没有经过人类实际阅读，agent 可能按照错误 milestones、遗漏的 **acceptance criteria** 或过时假设持续推进。自动化越强，坏计划造成的 token 浪费、代码偏移和 review 成本越高。

这条建议与前文的 feedback loop 一致：所有能影响 agent 行为的文本都属于 prompt surface。ticket、plan、AGENTS.md、skills、lint error、review comment 和 run book 都会改变执行方向。因此，高风险计划需要像代码一样接受 review；低风险 ticket 则依赖成熟 harness 提供足够上下文和 guardrails。

### 6.4 从 accepted code 到 *senior engineers as agents*

Ryan 说，written code 只有在被接受并推进产品后才有价值。这句话把第一章的 **code is free** 推向边界条件：代码生成便宜，代码接受仍然昂贵。接受意味着它通过测试、符合架构、满足用户旅程、没有明显可靠性缺口，并且对产品状态产生正向推进。

这里出现一个很重要的短语：**senior engineers as agents**。Ryan 提到，大家常说 senior engineers 会给出好的 code review；在 agent 时代，他也期待“资深工程师级别的 agent”承担类似职责。这个说法不应被理解为所有 agent 天然等同资深工程师。更准确的解释是：团队应当设计 reviewer

<sup>5</sup> 对应问答时间区间：00:39:28–00:40:32。



agents, 让它们以 reliability engineer、front-end architect、security reviewer、scalability reviewer 等 persona 读取同一套文档, 并只浮出足以阻塞合并的高价值问题。

#### 高信息问答: 代码是否是可丢弃的构建产物

**提问者 (00:41:30–00:42:17)** : Is code a disposable build artifact?

**Ryan:** Yes. Yes.

这句回答的含义需要加边界。生产代码仍需要维护; Ryan 在这里强调, 在某些系统中, 真正稳定的源资产可能是 spec、约束、测试和验收规则; 具体代码可以随着模型、工具和实现路径变化被重新生成、替换或迁移。只有被接受、可运行、可验证、可维护的代码, 才真正进入工程资产层。

### 6.5 *LLM as fuzzy compiler*: 把仓库上下文看作约束和优化 pass

#### *LLM as fuzzy compiler* 的定义

**LLM as fuzzy compiler** 是一种工程类比: 人类写下 spec、tickets、docs、rules、tests、constraints、optimization passes 和 acceptance criteria; LLM 像一个带不确定性的 compiler, 把这些输入转化为候选代码; CI、reviewer agents、QA 和人类验收再筛选出可接受产物。这里的“fuzzy”表示生成过程带概率性和模型行为差异, 需要外部验证闭环约束输出空间。

Ryan 用 **Symphony** 举例说明这层关系。Symphony 是他提到的 agent orchestrator; 在这个视角下, 团队可以发布一个由清晰 spec 定义的 library, 而代码只是从这个 spec 编译出的 artifact。把这个想法推广到整个代码库, Harness Engineering 中写入仓库的上下文、文档、lint、tests 和 reviewer agents, 都像 compiler 的限制条件和优化 pass: 它们不直接替代模型生成代码, 却持续限制“什么代码可以被生成并接受”。

#### LLVM、Craneflight 与模型替换

Ryan 把模型替换类比为 Rust 编译器中的 code generation backend 替换: 从 **LLVM** 换到 **Craneflight**, 最终机器指令可能不同, 但 Rust 语言规则、静态分析和优化 pass 仍然约束可接受输出。对应到 LLM, 换模型可能改变 patch 形状、命名习惯和实现路径; 仓库文档、测试、lint、reviewer agents 和验收标准承担“保持输出可接受”的作用。

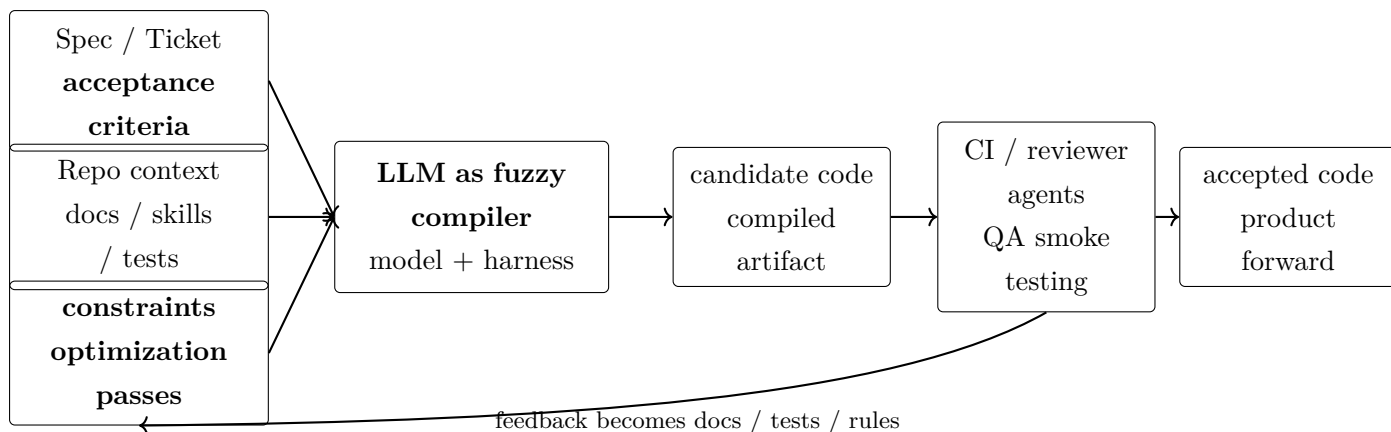


图 7: 把 Harness Engineering 压缩成 compiler 类比: spec、上下文和约束进入 LLM, 候选代码经过 CI、review 和 QA 后才成为 accepted artifact。该图根据 00:41:30–00:43:35 的问答重绘。

这个图也给出了全文的最短路径。第一章说明实现资源丰富; 第二章把团队经验写成 harness; 第三章把 Codex、skills、本地工具和 PR 串成协作面; 第四章把仓库改造成上下文机器; 第五章把 review 反馈变成自动 guardrails; 第六章把这些组件压缩为 **LLM as fuzzy compiler**: 人类写可执行约束, agent 生成候选实现, 验证系统决定哪些候选进入产品。

## 6.6 长期 autonomous engineering: 把软件工程外延纳入 agent 闭环

Ryan 在最后描绘的未来, 是给定一个 token budget、一季度、半年或一年的工作, 人类提供优先级排序、**success metrics** 和 **reliability metrics**, machines 持续推进产品, 而人类不需要一直手放在方向盘上。这里的关键仍然是“人类输入”与“持续验证”: 人类决定什么重要、怎样衡量成功、什么可靠性边界不可突破; agent 则在这些条件下长期执行、观察、修复和推进。

他还补充了一个很实际的经验: 团队从 early prototype 走向 internal alpha、internal beta、external alpha 时, 会发现软件工程的不同能力区域从零开始暴露出来。比如部署软件后, agent 做 **QA smoke testing** 的能力一开始很弱, 因为没有文档、没有工具、没有流程让它下载构建产物、启动应用、点击关键路径并证明 critical user journeys 得到验证。要实现长期 autonomous engineering, 团队需要让 agent 也能构建和使用这些工具。

这将软件工程从“写代码”扩展到更完整的运行系统:

- triaging user feedback 与 triaging pages: 把用户反馈和页面状态变成可分类、可执行的改进任务;
- **PII leaking in logs**: 持续检查生产日志, 避免隐私信息泄漏;
- Twitter vibes 与用户情绪: 观察外部反馈, 判断软件是否被真实用户喜欢和信任;
- **run books**: 为 user operations staff 写清高频问题的排查、缓解和升级流程;
- 从运营问题回流到代码: 把重复出现的人工处置需求迁移进产品、工具、测试或 guardrails, 使问题更早消失。

### *meta programming* 的工程含义

Ryan 所说的 **meta programming** 指向比生成代码更上层的工作: 工程师把流程、标准、验收、反馈、QA、日志检查和运营处置写成 agent 可执行的上层程序。agent 生成代码只是其中一环; 更高层的工作是定义整个软件工程系统怎样持续前进、怎样发现风险、怎样证明完

成。

## 6.7 全文结构化提炼：六章共同回答一个问题

整场演讲的抽象问题是：当 humans steer、agents execute 时，软件工程怎样重构。全文可以压缩为六个层次：

1. **稀缺资源变化**：**code is free** 表示实现、重构、删除的边际执行成本下降；human time、attention 和 context window 成为核心瓶颈。
2. **Harness 定义**：文档、rules、skills、lint、tests、reviewer agents、CI 和 error messages 共同向 agent 交付上下文。
3. **工作流入口**：Codex-first workflow 把 ticket、local observability stack、Chrome DevTools、custom ESLint rules 和 PR 协作连成执行路径。
4. **仓库形态**：package privacy、业务域边界、一致工具和 progressive disclosure 让 agent 在更小上下文中做出更一致的改变。
5. **反馈闭环**：garbage collection day 把 code review 中重复出现的 slop 归因为 context failure，再转成 docs、rules、tests 和 reviewer agents。
6. **终局类比**：**LLM as fuzzy compiler** 把 spec、constraints、optimization passes 和 acceptance criteria 编译为候选代码，再由验证系统筛出 accepted artifact。

这六个层次共同说明：Harness Engineering 的核心资产集中在团队持续把判断、标准、边界和验收写入工程环境的能力。模型会变，backend 会变，agent harness 会变；如果约束、指标和反馈闭环足够清晰，团队就能更稳地吸收模型能力提升。

## 6.8 本章小结

终章的核心结论是：Ryan 将 Harness Engineering 从“更高效地使用 coding agents”提升为一种长期 autonomous engineering 的工程观。人类给出优先级、上下文、预算、成功指标、可靠性指标和验收标准；agent 执行实现、验证、QA、运营辅助和持续修复；仓库里的约束和反馈机制像 compiler passes 一样限制输出空间。代码在这个模型中更接近 **code is a disposable build artifact**，但只有被测试、review、接受并推进产品的代码才真正产生价值。

### 实践清单

- 给每个重要 ticket 写清 **acceptance criteria**、关键用户旅程、风险边界和验证证据。
- 对高风险 **plan mode** 或 **exact plans** 输出采用单独 PR review，确认每一行计划都值得进入 rollout。
- 把重复 code review 意见归档为 context failure，并转成 docs、lint、tests、reviewer agents 或 CI checks。
- 建立 **success metrics** 与 **reliability metrics**，让 agent 能判断产品推进是否真实发生。
- 为 **QA smoke testing**、日志隐私检查、用户反馈 triage 和 **run books** 提供工具与文档，使软件工程外延也能进入 agent 闭环。
- 在换模型或换 harness 时，用测试、source-code checks、reviewer agents 和验收报告观察行为差异，类似观察 compiler backend 替换后的输出变化。

### 风险边界

- **LLM as fuzzy compiler** 是类比，不能提供传统编译器那种确定性 soundness；必须依赖测试、review、QA 和运行监控闭环。
- **roughly a third/third/third** 是 Ryan 的粗略经验，不应被包装成通用预算比例。
- 未读计划、过期文档、弱测试和模糊指标会被 agent 放大，长期 autonomous engineering 需要更严格的信息卫生。
- **code is a disposable build artifact** 只在 spec、constraints、tests 和 acceptance criteria 足够强时成立；缺少这些资产时，代码仍是团队理解系统的主要载体。
- **senior engineers as agents** 是目标状态和设计方向，不能当作对任意 agent 能力的无条件事实陈述。
- 软件工程外延进入 agent 闭环后，PII、用户运营、外部口碑和生产可靠性风险也进入自动化范围；权限、审计、回滚和人工接管机制必须提前设计。